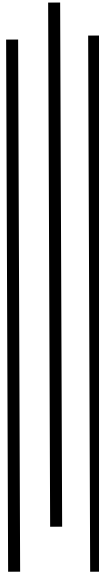


Tribhuvan University
Institute of Science and Technology



National College of Computer Studies

Bsc. CSIT
Second Semester

Project on Object Oriented Programming

Calorie Tracker

Submitted By

Name : Abhilekh Subedi, Ayush Tuladhar

Section : A

Roll : 1, 7

Submitted To

Janak Raj Joshi

Table of Contents

Abstract.....	i
1. Introduction.....	1
1.1. Background.....	1
1.2. Objective.....	1
2. Project Description.....	2
2.1. Purpose and Scope	2
2.2. Problem Description	2
2.3. Functionality and Features.....	3
3. Methodology	4
3.1. Development process	4
3.2. Tools and Resources	4
3.3. Development Environment	4
4. Results and Discussion	5
4.1. Presentation of Project Outcomes and Achievements	5
4.2. Comparison of Actual Results with Expected Outcomes	5
4.3. Discussion of Challenges Faced and Lessons Learned.....	6
5. Conclusion	7
6. Future Work	8
6.1. Suggestion for Future Enhancements	8
6.2. Ideas for Additional Improvements	8
7. References.....	9
8. Appendices.....	10
8.1. Source Code.....	10
8.2. Output	25

Abstract

This report elaborates on the progressions of a Calorie Tracker, an application software that essentially aids the user in tracking the number of calories and nutrients consumed for the day. While this application is written in C++, it has features for creating, displaying, and maintaining meal data. Users can save and track their meals of either breakfast, lunch, dinner, and snacks in association to the calories and proteins found in all those meals. The program also consists of a UserProfile engine that is a personal calorie and protein goals daily computation based on user-specific information, such as age, gender, weight, height, and activity level. It will also give alerts if the intakes become more or less than those set limits, giving the user more awareness of their nutritional intake. The program is modular; thus, it applies object-oriented principles such as inheritance and polymorphism for the reasons of scalability and maintainability. Such file handling features also make saving and accessing user data possible, thus the model is easy to use as well. The architecture of the program, core functionalities and details on implementation, as well as possible areas of future improvements, are well expounded in this report

1. Introduction

The Calorie Tracker project solves the challenge of tracking daily calorie and protein intake. Traditionally, one would use notepads or spreadsheets to track their daily diet, which is usually tedious and prone to errors with less scope for detailed analysis. This project automates calorie tracking, thereby improving efficiency and accuracy in dietary monitoring.

1.1. Background

Good diet management is all about striking a balance between calorie and nutrient intake. Conventional methods of diet tracking are very cumbersome and often lead to incomplete or inaccurate records, which make it difficult to achieve health goals. People today are looking more toward technology-driven solutions to aid in health and fitness; the Calorie Tracker effectively answers this demand with an easy and efficient tool. It brings together under one roof personalized goal setting, automated meal logging, and nutrient tracking to enable users to keep track of their diet.

1.2. Objective

This report was prepared to meet the following objectives:

- To provide an overview of the Calorie Tracker application.
- To explain the significance of automating dietary tracking.
- To discuss the system's functionality and features.
- To highlight the system's role in improving dietary management.
- To outline potential areas for future enhancements.

2. Project Description

2.1. Purpose and Scope

The calorie tracker is a software application that simplifies and automates the management of what one eats in tracking calorie and protein intake. It aims to help those people who want to maintain, lose, or gain weight by providing specifics on personal calorie and protein intakes based on specific user information, such as age, gender, weight, height, and activity level. Scopes of the application includes:

- Calculating and tracking daily calorie and protein intake.
- Logging meals categorized as breakfast, lunch, dinner, or snacks.
- Warning users when consumption exceeds set goals.
- Saving and retrieving user's data for long-term tracking and analysis.

2.2. Problem Description

The manual diet tracking process is extremely long-winded and also very error prone; it frequently leads to incomplete records and wrong calculations. Such inaccuracies preclude the achievement of dietary goals and, hence, compromise health. Lack of personalized dietary advice dilutes the otherwise overwhelming ability of these manual methods. The Calorie Tracker automates the entire process, thereby assuring accuracy while employing easy manual systems for effective dietary management. This is what I call an organization that gets everything right: - Attractions - Promoters - From manual diet tracking to the tedious and sometimes impossible processes that one can do without because it also has the greatest chance of being wrong, to the often incomplete and miscalculated records. Such inaccuracies do not allow an individual to meet his dietary goals and thus compromise health. Further, the lack of personalized dietary advice severely reduces the otherwise overpowering benefits of these manual methods. The Calorie Tracker automates the whole process such that accuracy is assured while employing easy manual systems to ensure effective dietary management.

2.3. Functionality and Features

Calorie tracker offers a comprehensive set of features and functionalities tailored to the needs of user. These include:

- **Personalized Goal Setting:** Facilitates an individual to customize the specific calorie and protein average target every day on basis of an individual health goal (e.g., lose, maintain or gain weight).
- **Meal Logging:** Enables users to log meals under predefined categories (breakfast, lunch, dinner, and snacks) along with calorie and protein values.
- **Automated Calculations:** Tracks and sums up total daily calorie and protein intake.
- **Data Storage and Retrieval:** Utilizes file handling to save and retrieve user data for long-term tracking.
- **Data Storage and Retrieval:** Utilizes file handling to save and retrieve user data for long-term tracking.

3. Methodology

3.1. Development process

- **Requirements Gathering:** First, I gathered all the information related to the problem and what is required to solve the problem
- **Design:** In the next step, I visualized a general system architecture such as the user interface and other system components. Then I sketched the design on a normal paper.
- **Implementation:** Now, I translated the design into code. This involved writing the code using C++ programming language in IDE called Dev-C++.
- **Testing:** After writing the code, I tested the application to check if all the components work properly. Through testing, I could make sure that the system meets all the requirements and solves the problem in an efficient manner.
- **Deployment:** Finally, after making sure that there are no bugs or errors, the system is ready to be used in the organization.

3.2. Tools and Resources

- **IDE (Integrated Development Environment):** The project used Dev-C++ IDE to write, edit, and debug code.
- **Compiler:** It used Mingw port of GCC (GNU Compiler Collection) as its compiler.
- **Libraries:** <iostream> for input and output operations, <fstream> for file handling to save and retrieve data, <string> for managing text input and meal attributes.

3.3. Development Environment

- **Platform:** The Calorie tracker is designed for desktop platforms, including Windows and macOS.
- **Operating System:** For development of the application, Windows Operating System was use

4. Results and Discussion

4.1. Presentation of Project Outcomes and Achievements

The development of the Calorie Tracker has yielded several significant outcomes, including:

- **Automation of Dietary Tracking:** The application successfully automates key tasks, such as meal logging, calorie and protein tracking, and personalized goal-setting.
- **Personalized Goal Management:** The system calculates user-specific calorie and protein goals based on input data (age, weight, height, gender, and activity level), enhancing its utility for diverse user needs.
- **Data Persistence:** File handling enables the application to save and retrieve meal records, ensuring long-term tracking.
- **User-Friendly Design:** The interactive menu-based interface simplifies navigation and improves usability, catering to users with varying levels of technical expertise.

4.2. Comparison of Actual Results with Expected Outcomes

The actual results obtained from the Calorie Tracker implementation closely match the predicted outcomes specified during the project design phase. Specifically:

- **Automated Tracking:** The system automates calorie and protein tracking as intended, minimizing manual calculations and errors.
- **Accurate Goal Warnings:** The program successfully notifies users when their daily intake exceeds the set calorie or protein limits, supporting effective dietary management.
- **Efficient Meal Logging:** Users can log and view meals by category (breakfast, lunch, dinner, snacks) without system delays, meeting performance expectations.

4.3. Discussion of Challenges Faced and Lessons Learned

Throughout the project, a number of issues surfaced that made for excellent learning opportunities:

- **Memory Management:** Ensure proper memory management for the dynamically created meal objects by thorough debugging to ensure correct memory allocation and deallocation.
- **User input handling:** Proper input validation for meal names and numeric values (e.g., calories, protein) was a primary goal in ensuring that no run-time errors were generated.
- **File Handling Limitations:** At first, the robust implementation of file operations was a problem with data persistence, but eventually, it was solved through testing iterations.
- **Time Constraints:** Other things such as the representation of visual data or advanced analytics cannot be achieved due to time constraints.

These challenges highlighted the importance of planning, testing, and iterative improvements in software development.

5. Conclusion

The effectiveness of automated diet tracking for calory and protein intake control in any individual is very neatly demonstrated by Calorie Tracker project. It encourages personalized goals, and helps the individual log meals in an application used in the field for weight loss, maintenance, or gain. Object-oriented programming principles such as inheritance and polymorphism enabled this facility to be modular, scalable and user-friendly. Additional advantages of the system include sending users alerts when exceeding preset goals and using file handling to track long-term data. So all put together, Calorie Tracker is technology that advances food tracking significantly, providing ingenious ways for people to keep track of their calories for health improvement. Further developments that can be carried out in this respect include advanced analytics, user interfaces, and additional categorization for food.

6. Future Work

6.1. Suggestion for Future Enhancements

- **Mobile Application Development:** Mobile app for the system will allow even higher accessibility for users, who would be able to log meals anywhere on the go and track progress easily.
- **Advanced Nutrient Tracking:** Additional nutrient parameters such as fat, carbohydrates, vitamins, and minerals can be introduced in future along with calories and protein to check for a more complete dietary goal.
- **Visual Data Representation:** Create charts or graphs that display the daily intake patterns and allow the viewer to visualize their progress.
- **Personalized Meal Recommendations:** An algorithm can be made that provides meal planning and recipe options based on user preferences and targeted objectives such as weight loss or muscle gain.

6.2. Ideas for Additional Improvements

In addition to future developments, there are various ideas for more features or changes that could further enhance the functioning and usability of the calorie tracker. Such as:

- **Support for Multiple Users:** This way multiple users can manage their profiles within the same application that is mostly beneficial for families or fitness groups tracking similar shared goals.
- **Linking with Fitness Devices:** Connect the app to fitness trackers or smartwatches so that data on physical activity can sync directly to the application and automatically adjust calorie goals.
- **Meal Database Integration:** Providing an external meal database for calorie and nutrient information to mostly consumed food items, hence minimizing input from users.

7. References

- [w3schools.com](https://www.w3schools.com)
- [programmiz.com](https://www.programmiz.com)
- [javapoint.com](https://www.javapoint.com)
- [cplusplus.com](https://www.cplusplus.com)
- [geeksforgeeks.org](https://www.geeksforgeeks.org)
- isocpp.org

8. Appendices

8.1. Source Code

```
#include<iostream>
#include<fstream>
#include<string.h>
#include<stdlib.h>
#include<windows.h>
using namespace std;

enum ActivityLevel {
    SEDENTARY,
    LIGHTLY_ACTIVE,
    MODERATELY_ACTIVE,
    VERY_ACTIVE,
    EXTRA_ACTIVE
};

#define MAX_MEALS 4
#define MAX_DAYS 7

// Cursor Position
int *posx,*posy;

void dash() {
    CONSOLE_SCREEN_BUFFER_INFO csbi;
    int width,i;
    if(!GetConsoleScreenBufferInfo(GetStdHandle(STD_OUTPUT_HANDLE),&csbi)) {
        cout<<"Error getting console screen buffer info.";
        return;
    }
    width=csbi.dwSize.X;
    cout<<endl;
    for (i=0;i<width;i++) {
        cout<<"-";
    }
    cout<<endl<<endl;
    fflush(stdout);
}

void gotoxy(int x,int y) {
    HANDLE hConsole=GetStdHandle(STD_OUTPUT_HANDLE);
    if(hConsole==INVALID_HANDLE_VALUE) {
        cout<<"Error getting console handle.";
        exit(1);
    }
}
```

```

    }
    COORD cursorPos;
    cursorPos.X=x;
    cursorPos.Y=y;
    if(!SetConsoleCursorPosition(hConsole,cursorPos)) {
        cout<<"Error setting console cursor position.";
        exit(1);
    }
}

char confirm(char s[],int *x,int *y) {
    char c;
    gotoxy(*x,*y);
    cout<<s;
    cin>>c;
    return(c);
}

bool openFile(fstream &file,char* filename,ios::openmode mode) {
    file.open(filename,mode);
    if (file.fail()) {
        cout<<"Error: Could not open file "<<filename<<endl;
        return false;
    }
    return true;
}

// Abstract base class for meals
class Meal {
protected:
    char name[50];
    double calories;
    double protein;
    int day; // Track which day the meal belongs to

public:
    Meal() {
        strcpy(name,"N/A");
        calories=0;
        protein=0;
        day=0;
    }

    Meal(const char* n, double c, double p, int d) {
        strncpy(name,n,sizeof(name)-1);
        name[sizeof(name)-1]='\0';
        calories=c;

```

```

        protein=p;
        day=d;
    }

    virtual void displayMeal() = 0; // Pure virtual function

    // Getters
    double getCalories() const { return calories; }
    double getProtein() const { return protein; }
    const char* getName() const { return name; }
    int getDay() const { return day; }

    // Setter for updating meal
    virtual void updateMeal(const char* newName, double newCalories, double newProtein) {
        strncpy(name, newName, sizeof(name) - 1);
        name[sizeof(name) - 1] = '\0';
        calories = newCalories;
        protein = newProtein;
    }

    virtual ~Meal() {}
};

// Derived meal type classes
class Breakfast : public Meal {
public:
    Breakfast() : Meal() {}
    Breakfast(const char* n, double c, double p, int d) : Meal(n, c, p, d) {}

    void displayMeal() {
        cout << "Day " << day << " - Breakfast: "
              << name << ", " << calories << " kCal, " << protein << " g\n";
    }
};

class Lunch : public Meal {
public:
    Lunch() : Meal() {}
    Lunch(const char* n, double c, double p, int d) : Meal(n, c, p, d) {}

    void displayMeal() {
        cout << "Day " << day << " - Lunch: "
              << name << ", " << calories << " kCal, " << protein << " g\n";
    }
};

class Dinner : public Meal {

```

```

public:
    Dinner() : Meal() {}
    Dinner(const char* n, double c, double p, int d) : Meal(n, c, p, d) {}

    void displayMeal() {
        cout << "Day " << day << " - Dinner: "
              << name << ", " << calories << " kCal, " << protein << " g\n";
    }
};

class Snacks : public Meal {
public:
    Snacks() : Meal() {}
    Snacks(const char* n, double c, double p, int d) : Meal(n, c, p, d) {}

    void displayMeal() {
        cout << "Day " << day << " - Snacks: "
              << name << ", " << calories << " kCal, " << protein << " g\n";
    }
};

class UserProfile {
private:
    char name[50], gender[6];
    double age, height, weight, goalCalorie, goalProtein;
    int x, y, activityChoice;
    ActivityLevel activityLevel;
    double calculateBMR() {
        double bmr;
        if(strcmp(gender, "Male") == 0)
            bmr = 10 * weight + 6.25 * height - 5 * age + 5;
        else
            bmr = 10 * weight + 6.25 * height - 5 * age - 161;
        return bmr;
    }
    double getActivityMultiplier() {
        switch(activityLevel) {
            case SEDENTARY: return 1.2;
            case LIGHTLY_ACTIVE: return 1.375;
            case MODERATELY_ACTIVE: return 1.55;
            case VERY_ACTIVE: return 1.725;
            case EXTRA_ACTIVE: return 1.9;
            default: return 1.2;
        }
    }
public:
    UserProfile() {

```



```

name[0]='\0';
age=0;
gender[0]='\0';
height=0;
weight=0;
goalCalorie=0;
goalProtein=0;
}
void setProfile() {
    x=35,y=1;
    system("cls");
    gotoxy(x,y);
    cout<<"User Profile Setup :";
    y+=2;
    gotoxy(x,y);
    cout<<"Enter your name : ";
    cin.ignore();
    cin.getline(name,sizeof(name));
    gotoxy(x,++y);
    cout<<"Enter age : ";
    cin>>age;
    cin.ignore();
    gotoxy(x,++y);
    cout<<"Enter gender (Male/Female) : ";
    cin.getline(gender,sizeof(gender));
    gotoxy(x,++y);
    cout<<"Enter height (cm) : ";
    cin>>height;
    gotoxy(x,++y);
    cout<<"Enter weight (kg) : ";
    cin>>weight;
    y+=2;
    gotoxy(x,y);
    cout<<"Activity Levels:";
    gotoxy(x,++y);
    cout<<"1. Sedentary (little to no exercise)";
    gotoxy(x,++y);
    cout<<"2. Light Activity (light exercise 1-3 days/week)";
    gotoxy(x,++y);
    cout<<"3. Moderate Activity (moderate exercise 3-5 days/week)";
    gotoxy(x,++y);
    cout<<"4. Very Active (hard exercise 6-7 days/week or physical job)";
    gotoxy(x,++y);cout<<
    "5. Custom";
    gotoxy(x,++y);
    cout<<"Select activity level : ";
    cin>>activityChoice;
}

```

```

    calculateGoals();
    saveProfile();
}

void calculateGoals() {
    int goalChoice;

    x=35,y=16;

    double bmr=calculateBMR();
    double activityMultiplier=getActivityMultiplier();
    switch(activityChoice) {
        case 1: activityLevel = SEDENTARY; break;
        case 2: activityLevel = LIGHTLY_ACTIVE; break;
        case 3: activityLevel = MODERATELY_ACTIVE; break;
        case 4: activityLevel = VERY_ACTIVE; break;
        case 5: activityLevel = EXTRA_ACTIVE; break;
        default: activityLevel = SEDENTARY;
    }
    double baseCalories=bmr*activityMultiplier;
    gotoxy(x,++y);

        cout<<"Goal Options:";
        gotoxy(x,++y);
        cout<<"1. Lose Weight";
        gotoxy(x,++y);
        cout<<"2. Maintain Weight";
        gotoxy(x,++y);
        cout<<"3. Gain Weight";
        gotoxy(x,++y);
        cout<<"4. Custom Goal";
        gotoxy(x,++y);

    cout<<"Select goal : ";
        cin>>goalChoice;

    switch(goalChoice) {
        case 1: goalCalorie=baseCalories-500; break;
        case 2: goalCalorie=baseCalories; break;
        case 3: goalCalorie=baseCalories+500; break;
        case 4:
            gotoxy(x,y);
            cout<<"Enter custom calorie goal : ";
            cin>>goalCalorie;
            break;
    }

    goalProtein=weight*0.8*2.205; // Standard protein calculation
    y+=2;

        gotoxy(x,y);
        cout<<"Your daily calorie goal is set to "<<goalCalorie<<" calories.\n";

    gotoxy(x,++y);

        cout<<"Your daily protein goal is set to "<<goalProtein<<" grams.\n";

```

```

        (*posy)=29;
    }

void saveProfile() {
    ofstream file("profile.txt",ios::binary);
    gotoxy(35,27);
    if (file.is_open()) {
        file.write((char*)this, sizeof(UserProfile));
        file.close();
        cout<<"Profile saved successfully!";
    } else {
        cout<<"Unable to save profile.";
    }
}

void viewProfile() {
    x=35,y=4;
    fstream file;
    openFile(file,"profile.txt",ios::in|ios::binary);
    if (file.is_open()) {
        system("cls");

        gotoxy(x,y);
        dash();

        y+=3;
        gotoxy(x,y);
        cout<<"\tUser Profile\t";
        gotoxy(x,++y);
        dash();

        UserProfile tempProfile;
        file.read((char*)&tempProfile,sizeof(UserProfile));
        y+=3;
        gotoxy(x,y);

        cout<<"Name : "<<tempProfile.name;
        gotoxy(x,++y);
        cout<<"Age : " <<tempProfile.age;
        gotoxy(x,++y);
        cout<<"Gender : " <<tempProfile.gender;
        gotoxy(x,++y);
        cout<<"Height : " <<tempProfile.height<<" cm";
        gotoxy(x,++y);
        cout<<"Weight: " <<tempProfile.weight<<" kg";
        gotoxy(x,++y);
        cout<<"Activity Level : " <<tempProfile.activityLevel;
        gotoxy(x,++y);
        cout<<"Daily Calorie Goal : " <<tempProfile.goalCalorie<<" kCal";
        gotoxy(x,++y);
        cout<<"Daily Protein Goal : " <<tempProfile.goalProtein<<" g";
        (*posy)+=2;
    }
}

```

```

        file.close();
    } else {
        gotoxy(35,22);
        cout << "No profile found. Please set up a profile first.\n";
        (*posy)=24;
    }
}

double getGoalCalorie() { return goalCalorie; }
double getGoalProtein() { return goalProtein; }
};

class CalorieTracker {
private:
    Meal* dailyMeals[MAX_DAYS][MAX_MEALS];
    int mealCounts[MAX_DAYS][x,y];
    UserProfile profile;
    double dailyTotalCalories[MAX_DAYS],dailyTotalProtein[MAX_DAYS];
public:
    CalorieTracker() {
        for (int i=0;i<MAX_DAYS;i++) {
            mealCounts[i]=0;
            dailyTotalCalories[i]=0;
            dailyTotalProtein[i]=0;
            for(int j=0;j<MAX_MEALS;j++) {
                dailyMeals[i][j]=NULL;
            }
        }
    }
    // Overloaded += operator for adding calories
    void operator+=(Meal &meal) {
        int day=meal.getDay()-1; // Convert to 0-index
        dailyTotalCalories[day]+=meal.getCalories();
        dailyTotalProtein[day]+=meal.getProtein();
    }
    void addMeal() {
        EnterDay:
            int day;
            char choice;
            system("cls");
            gotoxy(35,5);
            cout<<"Enter day (1-<<MAX_DAYS<<) to add meals : ";
            cin>>day;
            if(day<1 || day>MAX_DAYS) {
                gotoxy(35,7);
                cout<<"Invalid day.";
                (*posy)=9;
                choice=confirm("Try again? [Y/N] : ",posx,posy);
            }
        }
    }
};

```

```

        if(choice=='y' || choice=='Y')
            goto EnterDay;
    }
do {
    x=35,y=5;
    int mealType;
    char name[50];
    double calories, protein;
    Meal* newMeal=NULL;
    gotoxy(x,y+=2);
    cout<<"1. Breakfast";
    gotoxy(x,++y);
    cout<<"2. Lunch";
    gotoxy(x,++y);
    cout<<"3. Dinner";
    gotoxy(x,++y);
    cout<<"4. Snacks";
    gotoxy(x,++y);
    cout<<"Select meal type : ";
    cin>>mealType;
    cin.ignore();
    if(mealType<1 || mealType>4) {
        gotoxy(x,y+=2);
        cout<<"Invalid choice! Please select a valid meal type (1-4).";
        (*posy)=15;
        choice=confirm("Try again? [Y/N] : ",posx,posy);
        if(choice=='y' || choice=='Y') {
            system("cls");
            goto EnterMeal;
        }
    }
    gotoxy(x,y+=2);
    cout<<"Enter food name : ";
    cin.getline(name,sizeof(name));
    gotoxy(x,++y);
    cout<<"Enter calories & protein : ";
    cin>>calories>>protein;
    switch(mealType) {
        case 1: newMeal=new Breakfast(name,calories,protein,day); break;
        case 2: newMeal=new Lunch(name,calories,protein,day); break;
        case 3: newMeal=new Dinner(name,calories,protein,day); break;
        case 4: newMeal=new Snacks(name,calories,protein,day); break;
        default:
            cout<<"Invalid meal type.\n";
            if(newMeal) delete newMeal;
            continue;
    }
}

```

```

        // Add meal to the specific day
        dailyMeals[day-1][mealCounts[day-1]++]=newMeal;
        *this+=*newMeal;

        gotoxy(x,y+=2);
        cout<<"Meal added successfully!\n";

        // Check if maximum meals reached after adding
        if(mealCounts[day-1]>=MAX_MEALS) {
            cout<<"Maximum meals reached for Day "<<day;
            break;
        }
        gotoxy(x,y+=2);
        choice=confirm("Add another meal? [Y/N]: ",&x,&y);
        if(choice=='y' || choice=='Y') {
            system("cls");
            EnterMeal:
            gotoxy(35,5);
            cout<<"Day "<<day<<" :";

        }
        else
            (*posy)=20;
    } while(choice=='y' || choice=='Y');
}

void loadMealsFromFile() {
    system("cls");
    fstream file;
    char* fileName;
    cout<<"Enter filename to load meals from: ";
    cin>>fileName;
    if(openFile(file,fileName,ios::in)) {
        int type;
        Meal* meal=NULL;
        // Clear previous meals before loading
        for(int day=0;day<MAX_DAYS;day++) {
            mealCounts[day]=0; // Reset the meal count for each day
        }
        for(int day=0;day<MAX_DAYS;day++) {
            for(int i=0;i<MAX_MEALS;++i) {
                file.read((char*)&type,sizeof(type));
                switch(type) {
                    case 1: meal=new Breakfast(); break;
                    case 2: meal=new Lunch(); break;
                    case 3: meal=new Dinner(); break;
                    case 4: meal=new Snacks(); break;
                    default: continue;
                }
                file.read((char*)meal,sizeof(*meal));
            }
        }
    }
}

```

```

        dailyMeals[day][mealCounts[day]++]=meal;
    }
}
file.close();
cout<<"Meals loaded successfully from \""<<fileName<<"\"\\n";
}
else {
    cout<<"Error loading file.\\n";
}
}

void viewMeals() {

    loadMealsFromFile();
    for (int day = 0; day < MAX_DAYS; day++) {
        if (mealCounts[day] > 0) {
            cout << "Day " << day + 1 << "\\n";
            for (int i = 0; i < mealCounts[day]; ++i) {
                Meal* meal = dailyMeals[day][i];
                cout << "\\tMeal " << i + 1 << ": " << meal->getName() << ", Calories: " << meal->getCalories() << ", Protein: " << meal-
>getProtein() << "g\\n";
            }
        } else {
            cout << "Day " << day + 1 << ": No meals recorded.\\n";
        }
    }
    (*posx)=0;
    (*posy)=38;
}

void saveMealsToFile() {
    char fileName[100];
    x=35,y=22;
    gotoxy(x,y);
    cout<<"Enter filename to save meals: ";
    cin>>fileName;
    fstream file;
    if(openFile(file,fileName,ios::out)) {
        for(int day=0;day<MAX_DAYS;++day) {
            for (int i=0;i<mealCounts[day];i++) {
                Meal* meal=dailyMeals[day][i];
                int type=0;
                if ((Breakfast*)(meal)) type=1;
                else if ((Lunch*)(meal)) type=2;
                else if ((Dinner*)(meal)) type=3;
                else if ((Snacks*)(meal)) type=4;
                file.write((char*)&type,sizeof(type));
            }
        }
    }
}

```

```

        file.write((char*)meal,sizeof(*meal));
    }
}
file.close();
gotoxy(x,++y);
cout<<"Meals saved successfully to \""<<fileName<<"\"\\n";
} else {
    gotoxy(x,y+=2);
    cout<<"Error saving file.\\n";
}
(*posy)=25;
}

struct Totals {
    double calories[7] = {0}; // Total calories for each day
    double protein[7] = {0}; // Total protein for each day

    // Overload operator+= for the Totals structure
    void operator+=(const Meal &meal) {
        int day = meal.getDay() - 1; // Convert to 0-index
        calories[day] += meal.getCalories();
        protein[day] += meal.getProtein();
    }
};

void analyze() {
    x=35,y=1;
    system("cls");
    char fileName[100];
    gotoxy(x,y);
    cout<<"Enter filename to analyze meals: ";
    cin>>fileName;
    fstream file;
    openFile(file,fileName,ios::in);
    fstream user;
    openFile(user,"profile.txt",ios::in);
    user.read((char *)&profile,sizeof(UserProfile));
    Totals totals; // Instance of the Totals structure

    Meal* meal = NULL;
    int type;

    while (file.read((char*)&type, sizeof(type))) {
        switch (type) {
            case 1: meal = new Breakfast(); break;

```



```

        case 2: meal = new Lunch(); break;
        case 3: meal = new Dinner(); break;
        case 4: meal = new Snacks(); break;
        default: continue;
    }

    file.read((char*)meal, sizeof(*meal));

    // Use overloaded += operator to add meal data to totals
    totals += *meal;

    delete meal; // Clean up dynamically allocated memory
}

file.close();
// Display totals
for (int day = 0; day < 7; ++day) {
    gotoxy(x,y+=2);
    cout << "Day " << day + 1 << ": ";
    gotoxy(x,++y);
    cout<< "Calories consumed = " << totals.calories[day] << " / "<<profile.getGoalCalorie()<<" Kcal";
    gotoxy(x,++y);
    cout<< "Protein consumed = " << totals.protein[day] << " / "<<profile.getGoalProtein()<<" g";
}
(*posy)=31;
}

~CalorieTracker() {
    for (int day = 0; day < MAX_DAYS; day++) {
        for (int i = 0; i < mealCounts[day]; i++) {
            delete dailyMeals[day][i];
        }
    }
}

};

void displayMenu() {
    system("cls");
    int x=35,y=4;
    gotoxy(x,y);
    dash();
    y+=3;
    gotoxy(x,y);
    cout<<"\tCalorie Tracker \t";
    gotoxy(x,++y);

```

```

dash();
y+=3;
gotoxy(x,y);
cout<<"1. Set Up User Profile";
gotoxy(x,++y);
cout<<"2. View User Profile";
gotoxy(x,++y);
cout<<"3. Add Meals";
gotoxy(x,++y);
cout<<"4. Save Meals";
gotoxy(x,++y);
cout<<"5. View Meals";
gotoxy(x,++y);
cout<<"6. Analyze Meals";
gotoxy(x,++y);
cout<<"7. Exit";
}

int main() {
    UserProfile profile;
    CalorieTracker tracker;
    int x=35,y=20;
    char choice;
    posx=&x;
    posy=&y;
    do {
        displayMenu();
        choice=confirm("Which operation do you want to perform? [1-7] : ",&x,&y);
        switch(choice) {
            case '1':
                profile.setProfile();
                break;
            case '2':
                profile.viewProfile();
                break;
            case '3':
                tracker.addMeal();
                break;
            case '4':
                tracker.saveMealsToFile();
                break;
            case '5':
                tracker.viewMeals();
                break;
            case '6':
                tracker.analyze();
                break;

```

```

case '7':
    system("cls");
    gotoxy(x,4);

dash();
gotoxy(45,8);
printf("Thank you for using the program.");
gotoxy(x,10);
dash();
exit(0);
break;

default:
    y+=2;
    gotoxy(x,y);

printf("Invalid choice. Please choose from 1 - 7.");
*posy=23;

}
choice=confirm("Do you want to continue? [Y/N] : ",posx,posy);
(*posy)=20;
} while(choice=='y' || choice=='Y');
return 0;
}

```

8.2. Output

```
-----
|      Calorie Tracker      |
|-----|
1. Set Up User Profile
2. View User Profile
3. Add Meals
4. Save Meals
5. View Meals
6. Analyze Meals
7. Exit

Which operation do you want to perform? [1-7] : 1
```

Fig 1 : Home interface

```
User Profile Setup :

Enter your name : Ayush Tuladhar
Enter age : 19
Enter gender (Male/Female) : Male
Enter height (cm) : 155
Enter weight (kg) : 69

Activity Levels:
1. Sedentary (little to no exercise)
2. Light Activity (light exercise 1-3 days/week)
3. Moderate Activity (moderate exercise 3-5 days/week)
4. Very Active (hard exercise 6-7 days/week or physical job)
5. Custom
Select activity level : 3

Goal Options:
1. Lose Weight
2. Maintain Weight
3. Gain Weight
4. Custom Goal
Select goal : 3

Your daily calorie goal is set to 2382.5 calories.
Your daily protein goal is set to 121.716 grams.

Profile saved successfully!

Do you want to continue? [Y/N] : y_
```

Fig 2 : Option 1 (User profile set up)

```
-----  
|      User Profile      |  
-----  
  
Name : Ayush Tuladhar  
Age : 19  
Gender : Male  
Height : 155 cm  
Weight: 69 kg  
Activity Level : 2  
Daily Calorie Goal : 2382.5 kCal  
Daily Protein Goal : 121.716 g  
  
Do you want to continue? [Y/N] : y
```

Fig 3 : Option 2 (View user profile)

```
Enter day (1-7) to add meals : 1  
  
1. Breakfast  
2. Lunch  
3. Dinner  
4. Snacks  
Select meal type : 1  
  
Enter food name : Peanuts  
Enter calories & protein : 500 300  
  
Meal added successfully!  
  
Add another meal? [Y/N]: n  
  
Do you want to continue? [Y/N] : y
```

Fig 4 : Option 3 (Add meals)

```
-----
|      Calorie Tracker      |
|-----|

1. Set Up User Profile
2. View User Profile
3. Add Meals
4. Save Meals
5. View Meals
6. Analyze Meals
7. Exit

Which operation do you want to perform? [1-7] : 4

Enter filename to save meals: demo.txt
Meals saved successfully to "demo.txt"

Do you want to continue? [Y/N] : y_
```

Fig 5 : Option 4 (Save meals)

```
Enter filename to load meals from: demo.txt
Meals loaded successfully from "demo.txt"
Day 1:
    Meal 1: Peanuts, Calories: 500, Protein: 300g
    Meal 2: N/A, Calories: 0, Protein: 0g
    Meal 3: N/A, Calories: 0, Protein: 0g
    Meal 4: N/A, Calories: 0, Protein: 0g
Day 2:
    Meal 1: N/A, Calories: 0, Protein: 0g
    Meal 2: N/A, Calories: 0, Protein: 0g
    Meal 3: N/A, Calories: 0, Protein: 0g
    Meal 4: N/A, Calories: 0, Protein: 0g
Day 3:
    Meal 1: N/A, Calories: 0, Protein: 0g
    Meal 2: N/A, Calories: 0, Protein: 0g
    Meal 3: N/A, Calories: 0, Protein: 0g
    Meal 4: N/A, Calories: 0, Protein: 0g
Day 4:
    Meal 1: N/A, Calories: 0, Protein: 0g
    Meal 2: N/A, Calories: 0, Protein: 0g
    Meal 3: N/A, Calories: 0, Protein: 0g
    Meal 4: N/A, Calories: 0, Protein: 0g
Day 5:
    Meal 1: N/A, Calories: 0, Protein: 0g
    Meal 2: N/A, Calories: 0, Protein: 0g
    Meal 3: N/A, Calories: 0, Protein: 0g
    Meal 4: N/A, Calories: 0, Protein: 0g
Day 6:
    Meal 1: N/A, Calories: 0, Protein: 0g
    Meal 2: N/A, Calories: 0, Protein: 0g
    Meal 3: N/A, Calories: 0, Protein: 0g
    Meal 4: N/A, Calories: 0, Protein: 0g
Day 7:
    Meal 1: N/A, Calories: 0, Protein: 0g
    Meal 2: N/A, Calories: 0, Protein: 0g
    Meal 3: N/A, Calories: 0, Protein: 0g
    Meal 4: N/A, Calories: 0, Protein: 0g

Do you want to continue? [Y/N] : y
```

Fig 6 : Option 5 (View meals)

```
Enter filename to analyze meals: demo.txt

Day 1:
Calories consumed = 500 / 2382.5 Kcal
Protein consumed = 300 / 121.716 g

Day 2:
Calories consumed = 0 / 2382.5 Kcal
Protein consumed = 0 / 121.716 g

Day 3:
Calories consumed = 0 / 2382.5 Kcal
Protein consumed = 0 / 121.716 g

Day 4:
Calories consumed = 0 / 2382.5 Kcal
Protein consumed = 0 / 121.716 g

Day 5:
Calories consumed = 0 / 2382.5 Kcal
Protein consumed = 0 / 121.716 g

Day 6:
Calories consumed = 0 / 2382.5 Kcal
Protein consumed = 0 / 121.716 g

Day 7:
Calories consumed = 0 / 2382.5 Kcal
Protein consumed = 0 / 121.716 g

Do you want to continue? [Y/N] : _
```

Fig 7 : Option 6 (Analyze meals)

```
-----
Thank you for using the program.
-----
```

Fig 8: Option 7 (Exit)